Part 3: LangChain RAG

Document Loaders, Text Splitters, Embeddings, Vector Stores, Retrievers & RAG Chains

Prerequisites: Complete Part 1 (Foundations) and Part 2 (Core)
Concepts Covered: 50+
Reading Time: 5-6 hours
Hands-On: Build complete RAG pipeline for expense documentation

---

Table of Contents

11. Document Loaders - Ingesting Data
12. Text Splitters - Chunking Documents
13. Embeddings - Semantic Representation
14. Vector Stores - Similarity Search
15. Retrievers - Smart Context Retrieval
16. RAG Chains - Question Answering

Chapter 11: Document Loaders - Ingesting Data

11.1 What Are Document Loaders?

Document Loader = Reads files/data into LangChain format

Think of loaders as:

- 📄 **File Readers**: Load content from various sources
- 🔄 **Format Converters**: Convert to standard Document format
- 📦 **Metadata Extractors**: Capture source information

Problem: Different file formats

```
// Without loaders (manual parsing)
const fs = require('fs');
const pdf = require('pdf-parse');
const csv = require('csv-parser');

// Different logic for each format
if (file.endsWith('.pdf')) {
  const buffer = fs.readFileSync(file);
  const data = await pdf(buffer);
  return data.text;
}
else if (file.endsWith('.csv')) {
  // Different parsing logic...
```

```
}
// 😩 Tedious!
```

```
// With loaders (unified interface)
const loader = new PDFLoader(filePath);
const docs = await loader.load();  // Works for any loader!
```

11.2 Concept: Document Format

Document = Standard format (all loaders output this)

```
{
  pageContent: "This is the text content of the document...",
  metadata: {
    source: "/path/to/file.pdf",
    page: 1,
    createdAt: "2026-02-09",
    author: "Alice"
  }
}
```

Fields:

- **pageContent**: Main text content (string)
- **metadata**: Source info, page numbers, timestamps, etc. (object)

11.3 Concept: PDFLoader

PDFLoader = Load PDF files

```
import { PDFLoader } from "langchain/document_loaders/fs/pdf";

const loader = new PDFLoader("./documents/expense-policy.pdf");

// Load returns array of Documents (one per page)
const docs = await loader.load();

console.log(docs);
// [
//   {
//     pageContent: "Expense Policy\n\nAll expenses must be...",
//     metadata: { source: "./documents/expense-policy.pdf", pdf: { totalPages: 5
}, loc: { pageNumber: 1 } }
//   },
//   {
//     pageContent: "Eligible Expenses:\n- Transportation...",
//     metadata: { source: "./documents/expense-policy.pdf", pdf: { totalPages: 5
```

```
}, loc: { pageNumber: 2 } }
//   },
//   // ... pages 3-5
// ]

console.log(docs[0].pageContent);
// "Expense Policy\n\nAll expenses must be submitted..."
```

Configuration Options:

```
const loader = new PDFLoader(filePath, {
  splitPages: true,  // One document per page (default: true)
  // If false: One document for entire PDF

  pdfjs: () => import("pdfjs-dist/legacy/build/pdf.js")
  // Custom PDF.js version
});
```

11.4 Concept: TextLoader

TextLoader = Load plain text files

```
import { TextLoader } from "langchain/document_loaders/fs/text";

const loader = new TextLoader("./documents/faq.txt");
const docs = await loader.load();

console.log(docs);
// [
//   {
//     pageContent: "Q: How do I submit an expense?\nA: Log into the system...",
//     metadata: { source: "./documents/faq.txt" }
//   }
// ]
```

11.5 Concept: CSVLoader

CSVLoader = Load CSV files

```
import { CSVLoader } from "langchain/document_loaders/fs/csv";

// File: expenses.csv
// date,amount,category,description
// 2026-02-09,500,Food,Lunch
// 2026-02-08,300,Transport,Taxi

const loader = new CSVLoader("./data/expenses.csv");
```

```
const docs = await loader.load();

console.log(docs);
// [
//   {
//     pageContent: "date: 2026-02-09\namount: 500\ncategory: Food\ndescription:
Lunch",
//     metadata: { source: "./data/expenses.csv", line: 1 }
//   },
//   {
//     pageContent: "date: 2026-02-08\namount: 300\ncategory:
Transport\ndescription: Taxi",
//     metadata: { source: "./data/expenses.csv", line: 2 }
//   }
// ]
```

Configuration:

```
const loader = new CSVLoader(file, {
  column: "description",  // Only load specific column
  separator: ";",  // Custom delimiter (default: ",")
});
```

11.6 Concept: JSONLoader

JSONLoader = Load JSON files

```
import { JSONLoader } from "langchain/document_loaders/fs/json";

// File: categories.json
// [
//   { "name": "Food", "description": "Meals, groceries, restaurants" },
//   { "name": "Transport", "description": "Taxi, bus, train, fuel" }
// ]

const loader = new JSONLoader(
  "./data/categories.json",
  ["/name", "/description"]  // JSONPointer paths to extract
);

const docs = await loader.load();

console.log(docs);
// [
//   {
//     pageContent: "Food\nMeals, groceries, restaurants",
//     metadata: { source: "./data/categories.json", line: 0 }
//   },
//   {
```

```
//     pageContent: "Transport\nTaxi, bus, train, fuel",
//     metadata: { source: "./data/categories.json", line: 1 }
//   }
// ]
```

11.7 Concept: DirectoryLoader

DirectoryLoader = Load all files in directory

```
import { DirectoryLoader } from "langchain/document_loaders/fs/directory";
import { PDFLoader } from "langchain/document_loaders/fs/pdf";
import { TextLoader } from "langchain/document_loaders/fs/text";

// Directory structure:
// ./docs/
//   ├── policy.pdf
//   ├── faq.txt
//   └── guide.pdf

const loader = new DirectoryLoader(
  "./docs",
  {
    ".pdf": (path) => new PDFLoader(path),
    ".txt": (path) => new TextLoader(path)
  }
);

const docs = await loader.load();
// Returns documents from all PDF and TXT files
```

11.8 Real Example: ai-langx/ Document Loading

```
// File: src/rag/loaders/documentLoader.js
import { PDFLoader } from "langchain/document_loaders/fs/pdf";
import { CSVLoader } from "langchain/document_loaders/fs/csv";
import { TextLoader } from "langchain/document_loaders/fs/text";
import path from "path";

export const loadDocuments = async (filePath) => {
  const ext = path.extname(filePath).toLowerCase();

  try {
    let loader;

    // Select loader based on file extension
    switch (ext) {
      case '.pdf':
        loader = new PDFLoader(filePath, {
          splitPages: true  // One doc per page
```

```
        });
        break;

      case '.txt':
      case '.md':
        loader = new TextLoader(filePath);
        break;

      case '.csv':
        loader = new CSVLoader(filePath, {
          column: undefined  // Load all columns
        });
        break;

      default:
        throw new Error(`Unsupported file type: ${ext}`);
    }

    // Load documents
    const docs = await loader.load();

    // Add custom metadata
    const enrichedDocs = docs.map((doc, index) => ({
      ...doc,
      metadata: {
        ...doc.metadata,
        uploadedAt: new Date().toISOString(),
        documentId: generateDocumentId(filePath),
        chunkIndex: index
      }
    }));

    console.log(`[Loader] Loaded ${enrichedDocs.length} documents from
${filePath}`);

    return enrichedDocs;

  } catch (error) {
    console.error(`[Loader] Failed to load ${filePath}:`, error);
    throw error;
  }
};

// Usage in upload handler
// File: src/routes/upload.js
router.post('/upload/document', authMiddleware, upload.single('file'), async (req,
res) => {
  try {
    const filePath = req.file.path;

    // 1. Load documents
    const documents = await loadDocuments(filePath);

    // 2. Split into chunks (next chapter)
```

```
    const chunks = await splitDocuments(documents);

    // 3. Embed and store (later chapters)
    await storeInVectorDB(chunks);

    res.json({
      success: true,
      documentsLoaded: documents.length,
      chunksCreated: chunks.length
    });

  } catch (error) {
    res.status(500).json({
      error: 'Failed to upload document',
      message: error.message
    });
  }
});
```

11.9 Additional Loaders

WebLoader (Web Scraping)

```
import { CheerioWebBaseLoader } from "langchain/document_loaders/web/cheerio";

const loader = new CheerioWebBaseLoader(
  "https://example.com/expense-policy"
);

const docs = await loader.load();
// Loads webpage content
```

NotionLoader (Notion Pages)

```
import { NotionAPILoader } from "langchain/document_loaders/web/notion";

const loader = new NotionAPILoader({
  clientOptions: {
    auth: process.env.NOTION_API_KEY
  },
  id: "page-id-here",  // Notion page ID
  type: "page"
});

const docs = await loader.load();
```

GoogleDriveLoader (Google Docs)

```
import { GoogleDriveLoader } from "langchain/document_loaders/web/google_drive";

const loader = new GoogleDriveLoader({
  // Auth configuration
});

const docs = await loader.load();
```

11.10 Custom Loader (Database)

Create custom loader for non-standard sources

```
import { BaseDocumentLoader } from "langchain/document_loaders/base";

class ExpenseDatabaseLoader extends BaseDocumentLoader {
  constructor(dbClient, userId) {
    super();
    this.dbClient = dbClient;
    this.userId = userId;
  }

  async load() {
    // Fetch data from database
    const expenses = await this.dbClient.query(
      'SELECT * FROM expenses WHERE user_id = $1',
      [this.userId]
    );

    // Convert to Document format
    return expenses.rows.map(expense => ({
      pageContent: `
Expense #${expense.id}
Amount: ₹${expense.amount}
Category: ${expense.category}
Description: ${expense.description}
Date: ${expense.date}
      `.trim(),
      metadata: {
        source: 'database',
        expenseId: expense.id,
        userId: expense.user_id,
        createdAt: expense.created_at
      }
    }));
  }
}

// Usage
const loader = new ExpenseDatabaseLoader(dbClient, userId);
const docs = await loader.load();
```

☑ **You now understand Document Loaders!**

Chapter 12: Text Splitters - Chunking Documents

12.1 Why Split Documents?

Problem: Documents are too long for LLM context window

```
// Loaded document: 50,000 tokens (100 pages PDF)
// LLM context window: 4,000 tokens ❌ Won't fit!

// Solution: Split into smaller chunks
// Chunk 1: 500 tokens (pages 1-2)
// Chunk 2: 500 tokens (pages 3-4)
// ...
// Chunk 100: 500 tokens (pages 99-100)
```

Goals:

- ☑ Fit within LLM context
- ☑ Preserve semantic meaning
- ☑ Enable targeted retrieval

12.2 Concept: Chunk Size & Overlap

Chunk size = Maximum characters per chunk
Overlap = Characters shared between adjacent chunks

```
Original Text: "The expense policy requires all receipts. Receipts must be
submitted within 30 days. Late submissions are not accepted."

Chunk Size = 50, Overlap = 10

Chunk 1: "The expense policy requires all receipts. Rece"
                                                    ↓ overlap (10 chars)
Chunk 2:                                        "Receipts must be submitted within
30 days. La"

↓ overlap
Chunk 3:
"Late submissions are not accepted."
```

Why overlap?

- ☑ Prevents cutting sentences mid-word
- ☑ Preserves context across boundaries

- ☑ Improves retrieval (related info in multiple chunks)

Typical values:

- Chunk size: 500-1500 characters (depending on document type)
- Overlap: 50-200 characters (10-20% of chunk size)

12.3 Concept: RecursiveCharacterTextSplitter

Most common splitter (splits recursively by separators)

```
import { RecursiveCharacterTextSplitter } from "langchain/text_splitter";

const splitter = new RecursiveCharacterTextSplitter({
  chunkSize: 1000,  // Max 1000 characters per chunk
  chunkOverlap: 200  // 200 character overlap
});

const text = `
Expense Policy

All employees must submit expense reports monthly.

Eligible Expenses:
1. Transportation
2. Meals
3. Accommodation

Ineligible Expenses:
- Personal purchases
- Entertainment
`;

const chunks = await splitter.createDocuments([text]);

console.log(chunks);
// [
//   { pageContent: "Expense Policy\n\nAll employees must submit...", metadata: {}
},
//   { pageContent: "...Eligible Expenses:\n1. Transportation...", metadata: {} },
//   { pageContent: "...Ineligible Expenses:\n- Personal...", metadata: {} }
// ]
```

How it works (recursive splitting):

1. **Try splitting by paragraphs** (`\n\n`)
    - If chunk still too large → continue
2. **Try splitting by newlines** (`\n`)
    - If chunk still too large → continue

3. **Try splitting by sentences** (`. `)
    - If chunk still too large → continue
4. **Try splitting by words** (` `)
    - If chunk still too large → continue
5. **Split by characters** (last resort)

Result: Preserves structure as much as possible

12.4 Configuration Options

```
const splitter = new RecursiveCharacterTextSplitter({
  // Size limits
  chunkSize: 1000,  // Max characters per chunk
  chunkOverlap: 200,  // Overlap between chunks

  // Separators (default order)
  separators: ["\n\n", "\n", ". ", " ", ""],
  // Tries each in order, uses first that keeps chunk under limit

  // Length function (custom)
  lengthFunction: (text) => text.length,  // Default: character count
  // Could use token counter: (text) => encoder.encode(text).length

  // Keep separator
  keepSeparator: false  // Whether to include separator in chunk
});
```

12.5 Real Example: ai-langx/ Chunking

```
// File: src/rag/chunker.js
import { RecursiveCharacterTextSplitter } from "langchain/text_splitter";

export const splitDocuments = async (documents) => {
  // Create splitter
  const splitter = new RecursiveCharacterTextSplitter({
    chunkSize: 1000,  // 1000 chars ≈ 250 tokens
    chunkOverlap: 200,  // 20% overlap

    // Custom separators for expense documents
    separators: [
      "\n\n",  // Paragraphs
      "\n",  // Lines
      ". ",  // Sentences
      ", ",  // Clauses
      " ",  // Words
      ""  // Characters
```

```
    ],

    lengthFunction: (text) => text.length
  });

  try {
    // Split all documents
    const chunks = await splitter.splitDocuments(documents);

    // Add chunk metadata
    const enrichedChunks = chunks.map((chunk, index) => ({
      ...chunk,
      metadata: {
        ...chunk.metadata,
        chunkId: `chunk-${index}`,
        chunkSize: chunk.pageContent.length,
        totalChunks: chunks.length
      }
    }));

    console.log(`[Chunker] Created ${enrichedChunks.length} chunks`);
    console.log(`[Chunker] Avg chunk size: ${
      Math.round(enrichedChunks.reduce((sum, c) => sum + c.pageContent.length, 0)
/ enrichedChunks.length)
    } characters`);

    return enrichedChunks;

  } catch (error) {
    console.error('[Chunker] Failed to split documents:', error);
    throw error;
  }
};

// Usage
// File: src/routes/upload.js
router.post('/upload/document', authMiddleware, upload.single('file'), async (req,
res) => {
  const documents = await loadDocuments(req.file.path);

  // Split into chunks
  const chunks = await splitDocuments(documents);

  res.json({
    documentsLoaded: documents.length,
    chunksCreated: chunks.length,
    avgChunkSize: Math.round(chunks.reduce((sum, c) => sum + c.pageContent.length,
0) / chunks.length)
  });
});
```

12.6 Concept: CharacterTextSplitter (Simple)

Simpler splitter (just splits at fixed size, single separator)

```
import { CharacterTextSplitter } from "langchain/text_splitter";

const splitter = new CharacterTextSplitter({
  separator: "\n",  // Split on newlines only
  chunkSize: 1000,
  chunkOverlap: 200
});

const chunks = await splitter.createDocuments([text]);
```

Difference from Recursive:

- CharacterTextSplitter: Uses one separator (simple)
- RecursiveCharacterTextSplitter: Tries multiple separators (preserves structure better)

When to use:

- CharacterTextSplitter: Simple documents with clear line structure
- RecursiveCharacterTextSplitter: Complex documents (most cases)

12.7 Concept: TokenTextSplitter (Token-Based)

Splits by token count (not character count)

```
import { TokenTextSplitter } from "langchain/text_splitter";

const splitter = new TokenTextSplitter({
  chunkSize: 100,  // 100 tokens per chunk
  chunkOverlap: 20,  // 20 token overlap
  encodingName: "cl100k_base"  // OpenAI's tokenizer
});

const chunks = await splitter.splitText(text);
```

Why use tokens?

- ☑ Precise control (LLM pricing is per token)
- ☑ Matches LLM's internal processing

Why use characters instead?

- ☑ Faster (no tokenization needed)
- ☑ Good approximation (1 token ≈ 4 characters for English)

12.8 Concept: Markdown Splitter (Structure-Aware)

Splits Markdown based on headers

```
import { MarkdownTextSplitter } from "langchain/text_splitter";

const markdown = `
# Expense Policy

## Eligible Expenses

### Transportation
- Taxi
- Train

### Meals
- Breakfast
- Lunch

## Submission

Submit within 30 days.
`;

const splitter = new MarkdownTextSplitter({
  chunkSize: 500,
  chunkOverlap: 50
});

const chunks = await splitter.createDocuments([markdown]);

// Preserves header hierarchy
// Chunk 1: "# Expense Policy\n\n## Eligible Expenses\n\n### Transportation\n-
Taxi\n- Train"
// Chunk 2: "## Eligible Expenses\n\n### Meals\n- Breakfast\n- Lunch"
// Chunk 3: "## Submission\n\nSubmit within 30 days."
```

Benefits:

- ☑ Preserves document structure
- ☑ Keeps related sections together
- ☑ Maintains context (includes parent headers)

12.9 Splitting Best Practices

1. Choose Appropriate Chunk Size

```
// Too small (< 200 chars)
chunkSize: 100
// ❌ Loses context
// ❌ More chunks = slower retrieval
// Example: "expense reports must" → Incomplete thought

// Too large (> 2000 chars)
```

```
chunkSize: 3000
// ❌ Exceeds LLM context if multiple chunks retrieved
// ❌ Less precise retrieval

// Just right (500-1500 chars)
chunkSize: 1000  // ☑ Good default
```

2. Set Reasonable Overlap

```
// No overlap
chunkOverlap: 0
// ❌ Can split sentences/thoughts mid-way

// Too much overlap (> 30%)
chunkOverlap: 500  // with chunkSize: 1000
// ❌ Wastes storage
// ❌ Redundant retrieval

// Just right (10-20%)
chunkOverlap: 200  // 20% of 1000 ☑
```

3. Use Semantic Separators

```
// ☑ Good: Preserves structure
separators: ["\n\n", "\n", ". ", " "]

// ❌ Bad: Breaks mid-sentence
separators: [" "]  // Only spaces
```

4. Add Chunk Metadata

```
const chunks = await splitter.splitDocuments(documents);

// Enrich with useful metadata
const enrichedChunks = chunks.map((chunk, index, array) => ({
  ...chunk,
  metadata: {
    ...chunk.metadata,
    chunkIndex: index,
    totalChunks: array.length,
    chunkSize: chunk.pageContent.length,
    documentTitle: extractTitle(documents[0]),
    section: extractSection(chunk.pageContent)  // e.g., "Eligible Expenses"
  }
}));
```

12.10 Potential ai-langx/ Enhancement: Smart Chunking

Not yet implemented - Semantic chunking based on content

```
// File: src/rag/semanticChunker.js
import { RecursiveCharacterTextSplitter } from "langchain/text_splitter";
import { OpenAIEmbeddings } from "@langchain/openai";
import { cosineSimilarity } from "../utils/similarity.js";

export class SemanticChunker {
  constructor(embeddings) {
    this.embeddings = embeddings;
  }

  async split(text, options = {}) {
    const { maxChunkSize = 1000, similarityThreshold = 0.7 } = options;

    // 1. Split into sentences
    const sentences = text.split(/(?<=[.!?])\s+/);

    // 2. Embed each sentence
    const embeddings = await this.embeddings.embedDocuments(sentences);

    // 3. Group similar consecutive sentences
    const chunks = [];
    let currentChunk = [sentences[0]];
    let currentEmbedding = embeddings[0];

    for (let i = 1; i < sentences.length; i++) {
      const similarity = cosineSimilarity(currentEmbedding, embeddings[i]);
      const chunkLength = currentChunk.join(' ').length;

      // Add to current chunk if similar and under size limit
      if (similarity > similarityThreshold && chunkLength + sentences[i].length <
maxChunkSize) {
        currentChunk.push(sentences[i]);
        // Update embedding (average of chunk embeddings)
        currentEmbedding = averageEmbeddings([currentEmbedding, embeddings[i]]);
      } else {
        // Start new chunk
        chunks.push(currentChunk.join(' '));
        currentChunk = [sentences[i]];
        currentEmbedding = embeddings[i];
      }
    }

    // Add last chunk
    if (currentChunk.length > 0) {
      chunks.push(currentChunk.join(' '));
    }

    return chunks.map(chunk => ({ pageContent: chunk, metadata: {} }));
```

```
  }
}

// Usage
const chunker = new SemanticChunker(new OpenAIEmbeddings());
const chunks = await chunker.split(documentText);
// Chunks based on semantic similarity, not just size
```

☑ **You now understand Text Splitting!**

Chapter 13: Embeddings - Semantic Representation

13.1 What Are Embeddings?

Embedding = Numerical representation of text (vector of numbers capturing meaning)

Think of embeddings as:

- 🧭 **Semantic Coordinates**: Position in meaning-space
- 🔢 **Vector**: Array of numbers (e.g., 1536 dimensions for OpenAI)
- 🎯 **Similarity Measure**: Similar meanings → Similar vectors

Analogy: GPS Coordinates

```
Mumbai: [19.0760°, 72.8777°]
Delhi:  [28.7041°, 77.1025°]

Distance between Mumbai-Delhi: ~1400km (far apart)

Navi Mumbai: [19.0330°, 73.0297°]
Distance Mumbai-Navi Mumbai: ~35km (very close)

→ Close coordinates = Close locations
```

Embeddings work similarly:

```
"lunch" → [0.2, 0.8, 0.1, ..., 0.5]  (1536 numbers)
"dinner" → [0.21, 0.79, 0.12, ..., 0.52]  (very similar!)

"car" → [0.9, 0.1, 0.7, ..., 0.3]  (very different!)

→ Similar embeddings = Similar meanings
```

13.2 How Embeddings Work

Training (OpenAI/Google trained these models):

```
Input text → Neural Network → Output vector

"expense report" → Model → [0.23, 0.76, 0.12, ..., 0.41]
                                  ↑
                      1536 numbers (dimensions)
```

Key property: Semantically similar text → Similar vectors

```
"expense report"  → [0.23, 0.76, ...]
"spending summary" → [0.24, 0.74, ...]  ← Close!

"pizza recipe" → [0.89, 0.12, ...]  ← Far!
```

13.3 Concept: OpenAIEmbeddings

OpenAIEmbeddings = Use OpenAI's embedding models

```javascript
import { OpenAIEmbeddings } from "@langchain/openai";

// Create embeddings instance
const embeddings = new OpenAIEmbeddings({
  modelName: "text-embedding-3-small",  // Model
  openAIApiKey: process.env.OPENAI_API_KEY
});

// Embed single text
const vector = await embeddings.embedQuery("expense report");

console.log(vector);
// [0.234, 0.765, 0.123, ..., 0.456]  (1536 numbers)
console.log(vector.length);
// 1536 (dimensions)

// Embed multiple texts (batch)
const vectors = await embeddings.embedDocuments([
  "lunch expense",
  "taxi fare",
  "hotel bill"
]);

console.log(vectors);
// [
//   [0.2, 0.8, ...],  // "lunch expense" vector
//   [0.3, 0.7, ...],  // "taxi fare" vector
//   [0.25, 0.75, ...]  // "hotel bill" vector
// ]
```

13.4 Embedding Models

OpenAI Embedding Models:

Model	Dimensions	Cost	Use Case
text-embedding-3-small	1536	$0.02/1M tokens	☑ Most use cases (ai-langx/ uses this)
text-embedding-3-large	3072	$0.13/1M tokens	Higher accuracy needed
text-embedding-ada-002	1536	$0.10/1M tokens	Legacy (still good)

Configuration:

```
const embeddings = new OpenAIEmbeddings({
  modelName: "text-embedding-3-small",

  // Batch size (for embedDocuments)
  batchSize: 512,  // Embed 512 documents at once (API limit: 2048)

  // Retry on error
  maxRetries: 3,

  // Timeout
  timeout: 30000  // 30 seconds
});
```

13.5 Real Example: ai-langx/ Embeddings

```
// File: src/rag/embeddings.js
import { OpenAIEmbeddings } from "@langchain/openai";

export const createEmbeddings = () => {
  return new OpenAIEmbeddings({
    modelName: "text-embedding-3-small",
    openAIApiKey: process.env.OPENAI_API_KEY,

    // Performance tuning
    batchSize: 512,  // Process 512 docs at once
    maxRetries: 3,  // Retry failed requests
    timeout: 30000,

    // Strip newlines (recommended by OpenAI)
    stripNewLines: true
  });
};

// Usage: Embed chunks before storing
// File: src/routes/upload.js
router.post('/upload/document', authMiddleware, upload.single('file'), async (req,
res) => {
```

```
  // 1. Load documents
  const documents = await loadDocuments(req.file.path);

  // 2. Split into chunks
  const chunks = await splitDocuments(documents);

  // 3. Create embeddings
  const embeddingsInstance = createEmbeddings();

  // 4. Embed all chunks
  console.log(`[Embeddings] Embedding ${chunks.length} chunks...`);
  const startTime = Date.now();

  const vectors = await embeddingsInstance.embedDocuments(
    chunks.map(chunk => chunk.pageContent)
  );

  const duration = Date.now() - startTime;
  console.log(`[Embeddings] Embedded ${chunks.length} chunks in ${duration}ms`);
  console.log(`[Embeddings] Avg: ${Math.round(duration / chunks.length)}ms per
chunk`);

  // 5. Store in vector database (next chapter)
  await storeInVectorDB(chunks, vectors);

  res.json({
    success: true,
    chunksEmbedded: chunks.length,
    embeddingTime: duration
  });
});
```

13.6 Concept: Cosine Similarity

Cosine similarity = Measure how similar two vectors are (0 to 1)

```
// Calculate similarity
function cosineSimilarity(vecA, vecB) {
  // Dot product
  const dotProduct = vecA.reduce((sum, val, i) => sum + val * vecB[i], 0);

  // Magnitudes
  const magA = Math.sqrt(vecA.reduce((sum, val) => sum + val * val, 0));
  const magB = Math.sqrt(vecB.reduce((sum, val) => sum + val * val, 0));

  // Cosine similarity
  return dotProduct / (magA * magB);
}

// Example
const lunchVector = await embeddings.embedQuery("lunch expense");
```

```
const dinnerVector = await embeddings.embedQuery("dinner expense");
const taxiVector = await embeddings.embedQuery("taxi fare");

const similarity1 = cosineSimilarity(lunchVector, dinnerVector);
console.log("lunch vs dinner:", similarity1);  // 0.92 (very similar!)

const similarity2 = cosineSimilarity(lunchVector, taxiVector);
console.log("lunch vs taxi:", similarity2);  // 0.65 (somewhat related)
```

Similarity ranges:

- **0.9-1.0**: Nearly identical meaning
- **0.7-0.9**: Closely related
- **0.5-0.7**: Somewhat related
- **< 0.5**: Different topics

13.7 Embedding Use Cases

1. Semantic Search (Main use in RAG)

```
// User query
const query = "How do I submit meal expenses?";
const queryVector = await embeddings.embedQuery(query);

// Compare with document chunks
const doc1 = "Meal expenses require receipts...";
const doc2 = "Transportation reimbursement process...";

const vec1 = await embeddings.embedQuery(doc1);
const vec2 = await embeddings.embedQuery(doc2);

const sim1 = cosineSimilarity(queryVector, vec1);  // 0.88 (high!)
const sim2 = cosineSimilarity(queryVector, vec2);  // 0.42 (low)

// Return doc1 (more relevant to query)
```

2. Clustering (Group similar content)

```
// Embed all expenses
const expenses = ["lunch", "dinner", "taxi", "uber", "breakfast"];
const vectors = await embeddings.embedDocuments(expenses);

// Group by similarity
// → Group 1: lunch, dinner, breakfast (food)
// → Group 2: taxi, uber (transport)
```

3. Classification (Categorize text)

```
// Category examples
const categories = {
  Food: await embeddings.embedQuery("restaurant meal groceries"),
  Transport: await embeddings.embedQuery("taxi uber bus train"),
  Shopping: await embeddings.embedQuery("clothes gadgets purchases")
};

// Classify new expense
const expense = "Starbucks coffee";
const expenseVec = await embeddings.embedQuery(expense);

// Find most similar category
let bestCategory = null;
let bestSimilarity = 0;

for (const [category, categoryVec] of Object.entries(categories)) {
  const similarity = cosineSimilarity(expenseVec, categoryVec);
  if (similarity > bestSimilarity) {
    bestSimilarity = similarity;
    bestCategory = category;
  }
}

console.log(bestCategory);  // "Food" (coffee → food)
```

13.8 Embedding Best Practices

1. Batch Embeddings (Faster, cheaper)

```
// ❌ Bad: One at a time (slow)
for (const text of texts) {
  const vector = await embeddings.embedQuery(text);  // N API calls
}

// ☑ Good: Batch (fast)
const vectors = await embeddings.embedDocuments(texts);  // 1 API call (or few)
```

2. Cache Embeddings (Avoid re-computing)

```
// File: src/utils/embeddingCache.js
const embeddingCache = new Map();

export const getEmbeddingCached = async (text, embeddings) => {
  // Check cache
  if (embeddingCache.has(text)) {
    return embeddingCache.get(text);
  }
```

```
  // Compute and cache
  const vector = await embeddings.embedQuery(text);
  embeddingCache.set(text, vector);
  return vector;
};
```

3. Normalize Text Before Embedding

```
function normalizeText(text) {
  return text
    .toLowerCase()  // Case-insensitive
    .replace(/\s+/g, ' ')  // Normalize whitespace
    .trim();
}

const vector = await embeddings.embedQuery(normalizeText(userInput));
```

4. Use Appropriate Model

```
// Small documents (< 1000 chars): text-embedding-3-small
const embeddings = new OpenAIEmbeddings({
  modelName: "text-embedding-3-small"  // ☑ Fast, cheap
});

// Large documents or high precision needed: text-embedding-3-large
const embeddings = new OpenAIEmbeddings({
  modelName: "text-embedding-3-large"  // ☑ More accurate, slower
});
```

☑ **You now understand Embeddings!**

Chapter 14: Vector Stores - Similarity Search

14.1 What Are Vector Stores?

Vector Store = Database for embeddings (enables fast similarity search)

Think of vector stores as:

- 🗄 **Specialized Database**: Stores vectors (not rows/tables)
- 🔍 **Semantic Search**: Find similar vectors quickly
- ⚡ **Optimized**: Fast approximate nearest neighbor search

Regular Database vs Vector Database

```
Regular (SQL):
┌──────────┬──────────┬──────────┐
│ id       │ text     │ category │
├──────────┼──────────┼──────────┤
│ 1        │ "lunch"  │ "Food"   │
│ 2        │ "taxi"   │ "Transport"│
└──────────┴──────────┴──────────┘

Search: WHERE text LIKE '%lun%'  ← Exact match

Vector Store:
┌──────────┬──────────────────┬──────────────────┐
│ id       │ text             │ vector           │
├──────────┼──────────────────┼──────────────────┤
│ 1        │ "lunch"          │ [0.2, 0.8, ...] │
│ 2        │ "dinner"         │ [0.21, 0.79, ...]│ ← Similar to "lunch"!
│ 3        │ "taxi"           │ [0.9, 0.1, ...] │
└──────────┴──────────────────┴──────────────────┘

Search: Find similar to "meal" → Returns "lunch", "dinner" ← Semantic!
```

14.2 Concept: MemoryVectorStore (In-Memory)

MemoryVectorStore = Simple in-memory vector store (good for prototyping)

```javascript
import { MemoryVectorStore } from "langchain/vectorstores/memory";
import { OpenAIEmbeddings } from "@langchain/openai";

// Create embeddings instance
const embeddings = new OpenAIEmbeddings({
  modelName: "text-embedding-3-small"
});

// Create vector store from documents
const documents = [
  { pageContent: "Lunch at restaurant costs ₹500", metadata: { category: "Food" }
},
  { pageContent: "Taxi fare from airport is ₹300", metadata: { category:
"Transport" } },
  { pageContent: "Dinner at hotel was ₹800", metadata: { category: "Food" } },
  { pageContent: "Uber ride to office ₹250", metadata: { category: "Transport" } }
];

const vectorStore = await MemoryVectorStore.fromDocuments(
  documents,
  embeddings
);

// Behind the scenes:
// 1. Embeds each document (4 API calls or 1 batch)
```

```
// 2. Stores vectors in memory
// 3. Ready for search

console.log("Vector store created with", documents.length, "documents");
```

14.3 Similarity Search

Search for similar documents

```
// Search: "meal expenses"
const results = await vectorStore.similaritySearch("meal expenses", 2);
// Returns top 2 most similar documents

console.log(results);
// [
//   { pageContent: "Lunch at restaurant costs ₹500", metadata: { category: "Food"
} },
//   { pageContent: "Dinner at hotel was ₹800", metadata: { category: "Food" } }
// ]

// "meal" → semantically similar to "lunch" and "dinner"!
```

With similarity scores:

```
const resultsWithScores = await vectorStore.similaritySearchWithScore("meal
expenses", 3);

console.log(resultsWithScores);
// [
//   [
//     { pageContent: "Lunch at restaurant costs ₹500", metadata: {...} },
//     0.92  // ← Similarity score (0-1)
//   ],
//   [
//     { pageContent: "Dinner at hotel was ₹800", metadata: {...} },
//     0.89
//   ],
//   [
//     { pageContent: "Taxi fare from airport is ₹300", metadata: {...} },
//     0.45  // ← Less similar
//   ]
// ]
```

14.4 Filtered Search (Metadata)

Filter by metadata

```
// Only search Food category
const results = await vectorStore.similaritySearch(
  "expenses",
  2,
  { category: "Food" }  // ← Filter
);

console.log(results);
// [
//   { pageContent: "Lunch at restaurant...", metadata: { category: "Food" } },
//   { pageContent: "Dinner at hotel...", metadata: { category: "Food" } }
// ]
// ← Only Food results (Transport filtered out)
```

14.5 Real Example: ai-langx/ Vector Store

```
// File: src/rag/vectorStore.js
import { MemoryVectorStore } from "langchain/vectorstores/memory";
import { createEmbeddings } from "./embeddings.js";
import fs from "fs/promises";
import path from "path";

const VECTOR_STORE_PATH = "./data/vector-store.json";

// Global in-memory store
let vectorStore = null;

// Initialize vector store
export const initVectorStore = async () => {
  const embeddings = createEmbeddings();

  try {
    // Try loading from disk (if exists)
    if (await fileExists(VECTOR_STORE_PATH)) {
      console.log('[VectorStore] Loading from disk...');
      const serialized = await fs.readFile(VECTOR_STORE_PATH, 'utf-8');
      vectorStore = await MemoryVectorStore.fromExistingSerialized(
        JSON.parse(serialized),
        embeddings
      );
      console.log(`[VectorStore] Loaded ${await vectorStore.index.length}
vectors`);
    } else {
      // Create empty store
      console.log('[VectorStore] Creating new store...');
      vectorStore = new MemoryVectorStore(embeddings);
      console.log('[VectorStore] Empty store created');
    }
  } catch (error) {
    console.error('[VectorStore] Initialization failed:', error);
```

```
    // Fallback: empty store
    vectorStore = new MemoryVectorStore(embeddings);
  }

  return vectorStore;
};

// Add documents to store
export const addDocuments = async (documents) => {
  if (!vectorStore) {
    await initVectorStore();
  }

  try {
    console.log(`[VectorStore] Adding ${documents.length} documents...`);
    await vectorStore.addDocuments(documents);

    // Persist to disk
    await saveVectorStore();

    console.log(`[VectorStore] Added successfully`);
  } catch (error) {
    console.error('[VectorStore] Failed to add documents:', error);
    throw error;
  }
};

// Search vector store
export const searchVectorStore = async (query, k = 3, filter = null) => {
  if (!vectorStore) {
    await initVectorStore();
  }

  try {
    console.log(`[VectorStore] Searching for: "${query}" (top ${k})`);

    const results = filter
      ? await vectorStore.similaritySearch(query, k, filter)
      : await vectorStore.similaritySearch(query, k);

    console.log(`[VectorStore] Found ${results.length} results`);
    return results;
  } catch (error) {
    console.error('[VectorStore] Search failed:', error);
    throw error;
  }
};

// Persist to disk
export const saveVectorStore = async () => {
  try {
    const serialized = await vectorStore.serialize();
    await fs.writeFile(
      VECTOR_STORE_PATH,
```

```
      JSON.stringify(serialized, null, 2),
      'utf-8'
    );
    console.log('[VectorStore] Saved to disk');
  } catch (error) {
    console.error('[VectorStore] Failed to save:', error);
  }
};

// Clear store
export const clearVectorStore = async () => {
  vectorStore = new MemoryVectorStore(createEmbeddings());
  await saveVectorStore();
  console.log('[VectorStore] Cleared');
};

// Utility
async function fileExists(filePath) {
  try {
    await fs.access(filePath);
    return true;
  } catch {
    return false;
  }
}

// Initialize on module load
initVectorStore();
```

14.6 Usage in Routes

```
// File: src/routes/upload.js
import { addDocuments } from "../rag/vectorStore.js";
import { loadDocuments } from "../rag/loaders/documentLoader.js";
import { splitDocuments } from "../rag/chunker.js";

router.post('/upload/document', authMiddleware, upload.single('file'), async (req,
res) => {
  try {
    const filePath = req.file.path;

    // 1. Load documents
    console.log('[Upload] Loading documents...');
    const documents = await loadDocuments(filePath);

    // 2. Split into chunks
    console.log('[Upload] Chunking documents...');
    const chunks = await splitDocuments(documents);

    // 3. Add to vector store (embeds + stores)
    console.log('[Upload] Adding to vector store...');
```

```
      await addDocuments(chunks);

      res.json({
        success: true,
        documentsLoaded: documents.length,
        chunksCreated: chunks.length,
        message: 'Document uploaded and indexed successfully'
      });

    } catch (error) {
      console.error('[Upload] Error:', error);
      res.status(500).json({
        error: 'Failed to upload document',
        message: error.message
      });
    }
  });

  // File: src/routes/chat.js (RAG search)
  import { searchVectorStore } from "../rag/vectorStore.js";

  router.post('/ai/rag', authMiddleware, async (req, res) => {
    try {
      const { question } = req.body;

      // Search for relevant chunks
      const relevantDocs = await searchVectorStore(question, 3);

      // Build context from results
      const context = relevantDocs
        .map(doc => doc.pageContent)
        .join('\n\n---\n\n');

      // Generate answer with LLM
      const answer = await llm.invoke([
        {
          role: "system",
          content: `Answer based on this context:\n\n${context}`
        },
        {
          role: "user",
          content: question
        }
      ]);

      res.json({
        answer: answer.content,
        sources: relevantDocs.map(doc => doc.metadata)
      });

    } catch (error) {
      res.status(500).json({ error: error.message });
    }
  });
```

14.7 Persistent Vector Stores

MemoryVectorStore limitations:

- ❌ Lost on server restart (unless saved to file)
- ❌ Not suitable for large datasets (> 10,000 docs)
- ❌ No distributed access

Production alternatives:

Pinecone (Cloud, Managed)

```
import { PineconeStore } from "@langchain/pinecone";
import { Pinecone } from "@pinecone-database/pinecone";

const pinecone = new Pinecone({
  apiKey: process.env.PINECONE_API_KEY
});

const index = pinecone.Index("expense-docs");

const vectorStore = await PineconeStore.fromDocuments(
  documents,
  embeddings,
  { pineconeIndex: index }
);

// Search (same API)
const results = await vectorStore.similaritySearch(query, 5);
```

Pros: Fully managed, scales automatically, fast
Cons: Costs money, external dependency

Chroma (Open Source, Self-Hosted)

```
import { Chroma } from "@langchain/community/vectorstores/chroma";

const vectorStore = await Chroma.fromDocuments(
  documents,
  embeddings,
  {
    collectionName: "expense-docs",
    url: "http://localhost:8000"  // ChromaDB server
  }
);

const results = await vectorStore.similaritySearch(query, 5);
```

Pros: Open source, self-hosted, free
Cons: Requires setup, manage yourself

Postgres (pgvector) (Postgres Extension)

```
import { PGVectorStore } from "@langchain/community/vectorstores/pgvector";

const vectorStore = await PGVectorStore.fromDocuments(
  documents,
  embeddings,
  {
    postgresConnectionOptions: {
      host: 'localhost',
      port: 5432,
      database: 'mydb',
      user: 'user',
      password: 'password'
    },
    tableName: 'documents'
  }
);

const results = await vectorStore.similaritySearch(query, 5);
```

Pros: Uses existing Postgres, no new infrastructure
Cons: Requires pgvector extension, slower than specialized stores

14.8 Vector Store Best Practices

1. Batch Additions

```
// ❌ Bad: One at a time
for (const doc of documents) {
  await vectorStore.addDocuments([doc]);  // N embed calls
}

// ☑ Good: Batch
await vectorStore.addDocuments(documents);  // 1 batch embed call
```

2. Filter Before LLM

```
// ☑ Good: Use metadata filters
const results = await vectorStore.similaritySearch(
  query,
  3,
```

```
  { userId: req.user.id }  // Only user's docs
);
```

3. Set Appropriate k (number of results)

```
// Too few: Miss relevant context
k = 1  // ❌ Might not have enough context

// Too many: Exceeds LLM context, adds noise
k = 100  // ❌ Too much text for LLM

// Just right: Balance
k = 3-5  // ☑ Good default
```

4. Persist to Disk (MemoryVectorStore)

```
// Save after updates
await vectorStore.serialize  // Serialization method
.then(data => fs.writeFile('vector-store.json', JSON.stringify(data)));

// Load on startup
const data = JSON.parse(await fs.readFile('vector-store.json', 'utf-8'));
vectorStore = await MemoryVectorStore.fromExistingSerialized(data, embeddings);
```

☑ **You now understand Vector Stores!**

Chapter 15: Retrievers - Smart Context Retrieval

15.1 What Are Retrievers?

Retriever = Interface for fetching relevant documents (wrapper around vector store)

Think of retrievers as:

- 🔍 **Search Interface**: Standard way to query documents
- 🧠 **Smart Filtering**: Advanced retrieval strategies
- 🔗 **Composable**: Use in chains/agents easily

Vector Store vs Retriever

```
// Vector Store: Direct similarity search
const results = await vectorStore.similaritySearch(query, 3);

// Retriever: Standardized interface (used in chains)
const retriever = vectorStore.asRetriever({ k: 3 });
```

```
const results = await retriever.getRelevantDocuments(query);

// Same result, but retriever works with LangChain chains
```

15.2 Basic Retriever (from Vector Store)

```
import { MemoryVectorStore } from "langchain/vectorstores/memory";

// Create vector store
const vectorStore = await MemoryVectorStore.fromDocuments(
  documents,
  embeddings
);

// Convert to retriever
const retriever = vectorStore.asRetriever({
  k: 3,  // Return top 3 results
  searchType: "similarity",  // Or "mmr"
  filter: { category: "Food" }  // Optional metadata filter
});

// Use retriever
const docs = await retriever.getRelevantDocuments("meal expenses");

console.log(docs);
// [ {pageContent: "...", metadata: {...}}, ... ]
```

15.3 Search Types

1. Similarity Search (Default)

```
const retriever = vectorStore.asRetriever({
  searchType: "similarity",
  k: 3
});

// Returns top 3 most similar documents
const results = await retriever.getRelevantDocuments("lunch");
// Results: docs most similar to "lunch"
```

2. MMR (Maximal Marginal Relevance) (Diverse results)

```
const retriever = vectorStore.asRetriever({
  searchType: "mmr",
  k: 3,
  fetchK: 20,  // Fetch 20 candidates
```

```
  lambda: 0.5  // Balance: 0=diversity, 1=similarity
});

// Returns diverse results (not all similar to each other)
const results = await retriever.getRelevantDocuments("expenses");

// Without MMR: ["lunch ₹500", "dinner ₹800", "breakfast ₹300"]  ← All food!
// With MMR: ["lunch ₹500", "taxi ₹300", "hotel ₹2000"]  ← Diverse categories
```

MMR algorithm:

1. Fetch top `fetchK` similar docs (e.g., 20)
2. Select most similar doc (rank #1)
3. Select next doc that is:
   - Similar to query (controlled by `lambda`)
   - Dissimilar to already-selected docs (diversity)
4. Repeat until `k` docs selected

When to use:

- ☑ Broad questions ("tell me about expenses")
- ☑ Want diverse examples
- ❌ Specific questions ("how to submit meal expenses")

15.4 Real Example: ai-langx/ Retriever

```
// File: src/rag/retriever.js
import { getVectorStore } from "./vectorStore.js";

export const createRetriever = (options = {}) => {
  const {
    k = 3,  // Top 3 results
    searchType = "similarity",  // or "mmr"
    filter = null  // Optional metadata filter
  } = options;

  const vectorStore = getVectorStore();

  if (!vectorStore) {
    throw new Error('Vector store not initialized');
  }

  return vectorStore.asRetriever({
    k,
    searchType,
    ...(filter && { filter })
  });
};

// Usage in RAG handler
// File: src/handlers/ragQaHandler.js
```

```
import { createRetriever } from "../rag/retriever.js";

export const handleRAGQuestion = async (question, context = {}) => {
  const { userId, traceId } = context;

  // Create retriever
  const retriever = createRetriever({
    k: 3,
    search Type: "similarity",
    // Optional: Filter by user's documents only
    // filter: { userId }
  });

  // Retrieve relevant docs
  console.log(`[RAG] Retrieving docs for: "${question}"`);
  const relevantDocs = await retriever.getRelevantDocuments(question);

  console.log(`[RAG] Found ${relevantDocs.length} relevant documents`);

  // Build context string
  const context = relevantDocs
    .map((doc, index) => `[Document ${index + 1}]\n${doc.pageContent}`)
    .join('\n\n---\n\n');

  return {
    relevantDocs,
    context
  };
};
```

15.5 Advanced Retrievers

Multi-Query Retriever (Generate variations)

```
import { MultiQueryRetriever } from "langchain/retrievers/multi_query";

// Generates multiple query variations, searches with each
const retriever = MultiQueryRetriever.fromLLM({
  llm,
  retriever: vectorStore.asRetriever(),
  verbose: true
});

// User query: "How do I claim meal expenses?"
// LLM generates variations:
// 1. "meal expense claiming process"
// 2. "submit food reimbursement"
// 3. "restaurant expense approval"
// Searches with all 3, combines results

const docs = await retriever.getRelevantDocuments(
```

```
  "How do I claim meal expenses?"
);
```

Benefits:

- ☑ Better recall (finds more relevant docs)
- ☑ Handles varied phrasing

Drawbacks:

- ❌ Slower (multiple LLM + search calls)
- ❌ Higher cost

Contextual Compression Retriever (Filter irrelevant parts)

```
import { ContextualCompressionRetriever } from
"langchain/retrievers/contextual_compression";
import { LLMChainExtractor } from
"langchain/retrievers/document_compressors/chain_extract";

// Retrieves docs, then extracts only relevant parts
const compressor = LLMChainExtractor.fromLLM(llm);

const retriever = new ContextualCompressionRetriever({
  baseCompressor: compressor,
  baseRetriever: vectorStore.asRetriever()
});

const docs = await retriever.getRelevantDocuments(
  "meal expense limits"
);

// Before compression:
// "Expense Policy: All expenses must be approved. Transportation includes taxi
and uber. Meal expenses are capped at ₹500 per day. Hotel stays require..."

// After compression:
// "Meal expenses are capped at ₹500 per day."  ← Only relevant part!
```

Benefits:

- ☑ Shorter context (saves tokens)
- ☑ More focused answers

Drawbacks:

- ❌ Extra LLM call (slower)
- ❌ Might remove useful context

Ensemble Retriever (Combine multiple)

```
import { EnsembleRetriever } from "langchain/retrievers/ensemble";

// Combine vector search + keyword search
const retriever = new EnsembleRetriever({
  retrievers: [
    vectorStore.asRetriever(),  // Semantic search
    bm25Retriever  // Keyword search (BM25 algorithm)
  ],
  weights: [0.7, 0.3]  // 70% semantic, 30% keyword
});

const docs = await retriever.getRelevantDocuments(query);
```

Benefits:

- ☑ Best of both worlds (semantic + exact match)
- ☑ More robust

Use case: When you need both semantic understanding and exact keyword matches

15.6 Retriever in Chains

Retrievers integrate seamlessly with chains

```
import { RetrievalQAChain } from "langchain/chains";

// Create chain with retriever
const chain = RetrievalQAChain.fromLLM(
  llm,
  retriever  // ← Retriever handles document fetching
);

// Ask question (chain automatically retrieves context)
const result = await chain.call({
  query: "What are the meal expense limits?"
});

console.log(result.text);
// "Meal expenses are capped at ₹500 per day according to the company policy."
```

Behind the scenes:

1. Chain passes query to retriever
2. Retriever fetches relevant docs
3. Chain builds prompt with docs as context
4. LLM generates answer
5. Chain returns result

15.7 Custom Retriever

Create custom retrieval logic

```
import { BaseRetriever } from "@langchain/core/retrievers";

class HybridRetriever extends BaseRetriever {
  constructor(vectorStore, dbClient, userId) {
    super();
    this.vectorStore = vectorStore;
    this.dbClient = dbClient;
    this.userId = userId;
  }

  async _getRelevantDocuments(query) {
    // 1. Vector search (semantic)
    const vectorResults = await this.vectorStore.similaritySearch(query, 5);

    // 2. Database search (exact user data)
    const dbResults = await this.dbClient.query(
      'SELECT * FROM expenses WHERE user_id = $1 AND description ILIKE $2 LIMIT
5',
      [this.userId, `%${query}%`]
    );

    // 3. Combine results
    const dbDocs = dbResults.rows.map(row => ({
      pageContent: `${row.description} - ₹${row.amount} (${row.category})`,
      metadata: { source: 'database', expenseId: row.id }
    }));

    // 4. Deduplicate and return
    return [...vectorResults, ...dbDocs].slice(0, 5);
  }
}

// Usage
const retriever = new HybridRetriever(vectorStore, dbClient, userId);
const docs = await retriever.getRelevantDocuments("recent food expenses");
```

☑ **You now understand Retrievers!**

Chapter 16: RAG Chains - Question Answering

16.1 What Is RAG?

RAG = Retrieval-Augmented Generation (Retrieve context + Generate answer)

Think of RAG as:

- 📚 **Context Provider**: Fetch relevant information
- 🤖 **Answer Generator**: LLM uses context to answer

- ☑ **Grounded**: Less hallucination (answers based on documents)

Without RAG (LLM only)

```
User: "What's our meal expense limit?"
LLM: "I don't have information about your company's expense policy."
❌ No knowledge of company docs
```

With RAG

```
User: "What's our meal expense limit?"

Step 1: Retrieve relevant docs
        → "Meal expenses are capped at ₹500 per day"

Step 2: LLM generates answer with context
        → "According to your company policy, meal expenses are capped at ₹500 per
day."

☑ Accurate, grounded in company docs
```

16.2 RAG Flow

```
User Question: "How do I submit meal expenses?"
                        ↓
┌──────────────────────────────────────────────┐
│              1. RETRIEVE                      │
│  - Embed query                                │
│  - Search vector store                        │
│  - Fetch top 3 relevant chunks                │
└──────────────────────────────────────────────┘
                    │
                    ↓
         Relevant Documents:
         - "Meal expenses require receipt..."
         - "Submit via expense portal..."
         - "Approval needed from manager..."
                    ↓
┌──────────────────────────────────────────────┐
│              2. AUGMENT                       │
│  - Build prompt with documents as context     │
│  - "Based on:\n[Doc 1]...\n[Doc 2]...\        │
│    Answer: How do I submit meal expenses?"    │
└──────────────────────────────────────────────┘
                    │
                    ↓
┌──────────────────────────────────────────────┐
│              3. GENERATE                      │
│  - LLM processes prompt + context             │
```

```
│  - Generates grounded answer                    │
└──────────────────────────────────────────────────┘
                    │
                    ↓
        Answer: "To submit meal expenses:
        1. Keep receipts
        2. Log into expense portal
        3. Submit for manager approval"
```

16.3 Concept: RetrievalQAChain

RetrievalQAChain = Standard RAG chain (most common)

```
import { RetrievalQAChain } from "langchain/chains";
import { ChatOpenAI } from "@langchain/openai";

// 1. Create retriever
const retriever = vectorStore.asRetriever({ k: 3 });

// 2. Create LLM
const llm = new ChatOpenAI({
  modelName: "gpt-4o-mini",
  temperature: 0  // Deterministic (for factual answers)
});

// 3. Create RAG chain
const chain = RetrievalQAChain.fromLLM(llm, retriever, {
  returnSourceDocuments: true,  // Include sources in response
  verbose: true  // Log steps
});

// 4. Ask question
const result = await chain.call({
  query: "What are the meal expense limits?"
});

console.log(result);
// {
//   text: "Meal expenses are capped at ₹500 per day according to company
policy.",
//   sourceDocuments: [
//     { pageContent: "Meal expenses capped at ₹500...", metadata: {...} },
//     { pageContent: "All expenses require receipts...", metadata: {...} },
//     { pageContent: "Manager approval needed...", metadata: {...} }
//   ]
// }
```

16.4 Real Example: ai-langx/ RAG Q&A

```
// File: src/handlers/ragQaHandler.js
import { RetrievalQAChain } from "langchain/chains";
import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { createRetriever } from "../rag/retriever.js";
import { getTraceTags, getTraceMetadata } from "../config/langsmith.config.js";

export const handleRAGQuestion = async (question, context = {}) => {
  const { userId, traceId } = context;

  try {
    // 1. Create retriever
    const retriever = createRetriever({
      k: 3,  // Top 3 relevant chunks
      searchType: "similarity"
    });

    // 2. Create LLM
    const llm = new ChatOpenAI({
      modelName: "gpt-4o-mini",
      temperature: 0,  // Factual answers

      // LangSmith tracing
      tags: getTraceTags('rag-qa', userId),
      metadata: getTraceMetadata(traceId, userId, {
        feature: 'rag-qa',
        question
      })
    });

    // 3. Custom prompt
    const prompt = ChatPromptTemplate.fromMessages([
      ["system", `You are an AI assistant that answers questions about expense
policies and procedures.

Use ONLY the provided context to answer. If the answer isn't in the context, say
"I don't have information about that in the documents."

Be concise and helpful. Cite specific rules or procedures when relevant.

Context:
{context}
`],
      ["human", "{question}"]
    ]);

    // 4. Create RAG chain
    const chain = RetrievalQAChain.fromLLM(llm, retriever, {
      prompt,
      returnSourceDocuments: true,
      verbose: process.env.NODE_ENV === 'development'
    });
```

```
    // 5. Execute chain
    console.log(`[RAG] Processing question: "${question}"`);
    const startTime = Date.now();

    const result = await chain.call({ query: question });

    const duration = Date.now() - startTime;
    console.log(`[RAG] Answered in ${duration}ms`);

    // 6. Format response
    return {
      answer: result.text,
      sources: result.sourceDocuments.map(doc => ({
        content: doc.pageContent.substring(0, 200) + '...',  // Preview
        metadata: doc.metadata
      })),
      processingTime: duration
    };

  } catch (error) {
    console.error('[RAG] Error:', error);
    throw error;
  }
};

// Usage in route
// File: src/routes/chat.js
router.post('/ai/rag', authMiddleware, async (req, res) => {
  try {
    const { question } = req.body;
    const userId = req.user.userId;
    const traceId = generateTraceId();

    const result = await handleRAGQuestion(question, { userId, traceId });

    res.json({
      answer: result.answer,
      sources: result.sources,
      metadata: {
        processingTime: result.processingTime,
        sourcesUsed: result.sources.length
      }
    });

  } catch (error) {
    res.status(500).json({
      error: 'Failed to process question',
      message: error.message
    });
  }
});
```

16.5 Custom RAG Prompt

Customize system instructions

```
const prompt = ChatPromptTemplate.fromMessages([
  ["system", `You are an expense policy assistant.

RULES:
1. Answer ONLY using provided context
2. If not in context, say "Not found in documents"
3. Be concise (2-3 sentences max)
4. Include specific amounts/numbers when relevant

Context:
{context}
`],
  ["human", "{question}"]
]);

const chain = RetrievalQAChain.fromLLM(llm, retriever, { prompt });
```

16.6 Concept: ConversationalRetrievalQAChain (With Memory)

RAG + conversation memory (remembers chat history)

```
import { ConversationalRetrievalQAChain } from "langchain/chains";
import { BufferMemory } from "langchain/memory";

// Create memory
const memory = new BufferMemory({
  memoryKey: "chat_history",
  returnMessages: true
});

// Create conversational RAG chain
const chain = ConversationalRetrievalQAChain.fromLLM(
  llm,
  retriever,
  { memory }
);

// Conversation
await chain.call({
  question: "What's the meal expense limit?"
});
// "₹500 per day"

await chain.call({
  question: "What about transportation?"
});
```

```
// "Transportation expenses are reimbursed..."

await chain.call({
  question: "Can you remind me about the meal limit?"
});
// "I mentioned earlier that meal expenses are capped at ₹500 per day."
// ↑ Remembers previous answer!
```

16.7 RAG Best Practices

1. Optimize Chunk Size for Retrieval

```
// Too small: Poor context
chunkSize: 200  // ❌ Fragments sentences

// Too large: Less precise retrieval
chunkSize: 3000  // ❌ Multiple topics per chunk

// Just right
chunkSize: 800-1200  // ☑ Complete thoughts, good retrieval
```

2. Set Appropriate k (Retrieved Docs)

```
// Too few: Miss important context
k: 1  // ❌ Might not have enough info

// Too many: Exceeds context, adds noise
k: 20  // ❌ Too much for LLM

// Just right
k: 3-5  //  ☑ Balance coverage and focus
```

3. Use Low Temperature for Factual QA

```
const llm = new ChatOpenAI({
  temperature: 0  // ☑ Deterministic, factual
});

// vs

const llm = new ChatOpenAI({
  temperature: 0.7  // ❌ Creative, might add unsupported info
});
```

4. Include Sources in Response

```
const chain = RetrievalQAChain.fromLLM(llm, retriever, {
  returnSourceDocuments: true  // ☑ Always enable
});

// Response includes sources for verification
// User can check答案 against source documents
```

5. Handle "Not Found" Gracefully

```
const prompt = ChatPromptTemplate.fromMessages([
  ["system", `Answer using provided context.

If answer NOT in context, respond EXACTLY:
"I don't have information about that in the uploaded documents."

Do NOT make up information.

Context:
{context}
`],
  ["human", "{question}"]
]);
```

16.8 Potential ai-langx/ Enhancements

Citation RAG (Quote sources)

```
const prompt = ChatPromptTemplate.fromMessages([
  ["system", `Answer using context and cite sources.

Format:
"[Answer] [Source: Document X]"

Example:
"Meal expenses are capped at ₹500 per day. [Source: Expense Policy, Page 3]"

Context:
{context}
`],
  ["human", "{question}"]
]);
```

Multi-Step RAG (Complex questions)

```
// User: "Compare meal and transport expense limits"
// Step 1: Retrieve "meal expense" docs
// Step 2: Retrieve "transport expense" docs
// Step 3: Compare and synthesize answer
```

☑ **You now understand RAG!**

Part 3 Summary

Concepts Covered: 50+

Document Loaders (10+ concepts)

- ☑ What loaders are and standard Document format
- ☑ PDFLoader, TextLoader, CSVLoader, JSONLoader
- ☑ DirectoryLoader for batch loading
- ☑ WebLoader, NotionLoader, GoogleDriveLoader
- ☑ Custom loaders (database, API)
- ☑ Real ai-langx/ document loading implementation

Text Splitters (10+ concepts)

- ☑ Why split documents (context window limits)
- ☑ Chunk size and overlap concepts
- ☑ RecursiveCharacterTextSplitter (recommended)
- ☑ CharacterTextSplitter (simple)
- ☑ TokenTextSplitter (token-based)
- ☑ MarkdownTextSplitter (structure-aware)
- ☑ Splitting best practices
- ☑ Semantic chunking strategies

Embeddings (10+ concepts)

- ☑ What embeddings are (semantic vectors)
- ☑ OpenAIEmbeddings and model options
- ☑ embedQuery vs embedDocuments
- ☑ Cosine similarity
- ☑ Embedding use cases (search, clustering, classification)
- ☑ Batch embedding for performance
- ☑ Caching strategies
- ☑ Text normalization

Vector Stores (10+ concepts)

- ☑ What vector stores are (databases for embeddings)
- ☑ MemoryVectorStore (in-memory)
- ☑ similaritySearch and similaritySearchWithScore

- ☑ Metadata filtering
- ☑ Persistence (save/load)
- ☑ Production stores (Pinecone, Chroma, pgvector)
- ☑ Real ai-langx/ vector store implementation
- ☑ Best practices (batching, filtering, k selection)

Retrievers (10+ concepts)

- ☑ What retrievers are (search interface)
- ☑ Basic retriever from vector store
- ☑ Search types (similarity, MMR)
- ☑ MMR for diversity
- ☑ Multi-Query retriever
- ☑ Contextual Compression retriever
- ☑ Ensemble retriever
- ☑ Custom retriever implementation
- ☑ Retriever in chains

RAG Chains (10+ concepts)

- ☑ What RAG is (Retrieval-Augmented Generation)
- ☑ RAG flow (Retrieve → Augment → Generate)
- ☑ RetrievalQAChain
- ☑ Custom RAG prompts
- ☑ ConversationalRetrievalQAChain (with memory)
- ☑ Returning source documents
- ☑ Real ai-langx/ RAG Q&A implementation
- ☑ RAG best practices
- ☑ Citation and multi-step RAG patterns

Complete Hands-On Exercise: Build Full RAG Pipeline

Goal: Create end-to-end RAG system for expense policy Q&A

Step 1: Load Documents

```
import { PDFLoader } from "langchain/document_loaders/fs/pdf";

const loader = new PDFLoader("./expense-policy.pdf");
const documents = await loader.load();
console.log(`Loaded ${documents.length} pages`);
```

Step 2: Split into Chunks

```
import { RecursiveCharacterTextSplitter } from "langchain/text_splitter";
```

```
const splitter = new RecursiveCharacterTextSplitter({
  chunkSize: 1000,
  chunkOverlap: 200
});

const chunks = await splitter.splitDocuments(documents);
console.log(`Created ${chunks.length} chunks`);
```

Step 3: Create Vector Store

```
import { MemoryVectorStore } from "langchain/vectorstores/memory";
import { OpenAIEmbeddings } from "@langchain/openai";

const embeddings = new OpenAIEmbeddings({
  modelName: "text-embedding-3-small"
});

console.log('Embedding chunks...');
const vectorStore = await MemoryVectorStore.fromDocuments(
  chunks,
  embeddings
);

console.log('Vector store created!');
```

Step 4: Create Retriever

```
const retriever = vectorStore.asRetriever({
  k: 3,
  searchType: "similarity"
});
```

Step 5: Create RAG Chain

```
import { RetrievalQAChain } from "langchain/chains";
import { ChatOpenAI } from "@langchain/openai";

const llm = new ChatOpenAI({
  modelName: "gpt-4o-mini",
  temperature: 0
});

const chain = RetrievalQAChain.fromLLM(llm, retriever, {
  returnSourceDocuments: true
});
```

Step 6: Ask Questions

```
const questions = [
  "What's the meal expense limit?",
  "How do I submit expenses?",
  "When do I need manager approval?"
];

for (const question of questions) {
  console.log(`\nQ: ${question}`);

  const result = await chain.call({ query: question });

  console.log(`A: ${result.text}`);
  console.log(`Sources: ${result.sourceDocuments.length} documents`);
}
```

Expected Output

```
Loaded 10 pages
Created 45 chunks
Embedding chunks...
Vector store created!

Q: What's the meal expense limit?
A: Meal expenses are capped at ₹500 per day according to company policy.
Sources: 3 documents

Q: How do I submit expenses?
A: To submit expenses: 1) Keep all receipts, 2) Log into the expense portal, 3)
Submit expense report by end of month, 4) Await manager approval.
Sources: 3 documents

Q: When do I need manager approval?
A: Manager approval is required for all expenses above ₹1,000 and for all travel-
related expenses regardless of amount.
Sources: 3 documents
```

Challenge Extensions

1. **Add conversation memory** (remember previous questions)
2. **Implement citation** (quote source page numbers)
3. **Add metadata filters** (filter by document type, date)
4. **Try MMR search** (get diverse results)
5. **Persist vector store** (save/load from disk)

Continue to Part 4: PART_4_LANGCHAIN_ADVANCED.md (LCEL, Runnables, Advanced Patterns - 50+ concepts)

Or jump to: PART_5_LANGGRAPH.md (StateGraph Workflows - 40+ concepts)